CodeGuardian AI for Laravel

Executive Summary · Analyze · Improve · Validate · Report

PREPARED BY Muhammad Bilal · Muhammad Suleman · Abdullah bin Yousaf

CodeGuardian AI is a production-grade static analysis, security audit, automated refactoring, and reporting platform for Laravel (PHP) and Flutter (Dart). It installs as a Composer package and runs from artisan commands or a built-in web dashboard — **offline, free, and deterministic by default**, with an optional AI engine for intent-aware review.

> **No API key. No internet. No cost. 100% embedded.** The default static engine performs all core analysis for free; AI mode (OpenAI · Claude · Gemini) is an optional upgrade for richer, natural-language explanations and deep refactors.

20	6	6
ARTISAN COMMANDS	HEALTH DIMENSIONS	REPORT FORMATS

THE PROBLEM & THE APPROACH

Why it exists

Most teams find architectural debt, security holes, and performance traps too late — in production or in painful reviews. CodeGuardian shifts that discovery *left*: it scans the codebase deterministically, scores health across six dimensions, explains every finding with a concrete fix, and can **safely apply many fixes automatically** behind a test-verified safety net that auto-rolls back any regression.

END-TO-END PIPELINE

The core workflow

1	2	3	4	5
Analyze	**Test-first**	**Refactor**	**Validate**	**Report**
Scan code; produce findings, scores & a risk rating.	Generate PHPUnit stubs *before* any change.	Deterministic + AI fixes, one file at a time.	Re-run tests; auto-rollback on regression.	Console, HTML, JSON, MD, SARIF, JUnit.

GRADED INSIGHT

Six-dimension scorecard

Every analysis grades the codebase 0–100 (with a letter grade and reasoning) across **Architecture**, **Security**, **Performance**, **Maintainability**, **Testability**, and **Reliability** — plus an explainable **risk score** weighted by severity × confidence. Security findings are mapped to OWASP Top 10 (2021), a CWE id, an indicative CVSS band, confidence, impact, effort, and breaking-change risk.

WHAT YOU GET

Key capabilities

Capability	What it does
Static engine	Architecture, security, performance, tech-debt & database analyzers — offline & deterministic.
Foolproof refactor	Test → refactor → verify loop with auto-rollback and syntax-validated writes.
Auto-fixes	No-AI mechanical fixes: remove `dd()`, `all()`→`validated()`, add return types.
AI engine	Optional OpenAI/Claude/Gemini review + diff-only mode for cheap PR review.

Capability	What it does
Premium CLI	Live multi-stage pipeline: spinners, ETA, per-file ticks, stats card; CI-safe plain mode.
Web dashboard	Run from the browser, live progress, history, insights trends & findings explorer.
Performance	Parallel analyzers, incremental git-scoped scans, AI + static result caching.
Extensible	Rule enable/severity, presets, custom regex rules, PHPStan/Psalm/Clover ingestion.

FITS YOUR TOOLCHAIN

CI/CD & reporting

Continuous integration

- **SARIF 2.1.0** upload for GitHub/GitLab/Azure code scanning.
- **Baseline / diff** — fail only on newly introduced findings.
- **Quality gates**, inline PR annotations, PR/MR comments.
- `codeguardian:doctor` preflight + JUnit test panels.

Command groups (20 total)

- **Analyze:** analyze, security, performance, generate-tests.
- **Improve:** refactor, fix.
- **Insight:** graph, trend, test-impact, rules, explain.
- **Ops:** init, doctor, audit, watch, review, notify, config-check.

BUILT FOR

Who it is for

Engineering teams & leads

CI health gating, PR-review automation, six-dimension scorecards, dependency graphs, and trend tracking.

Security engineers & developers

OWASP/CWE-mapped findings, CVE audits, SARIF output, a fast inner-loop watcher, and a friendly local dashboard.

AT A GLANCE

Stack & footprint

PHP ^8.1 · Laravel 9 / 10 / 11 · Flutter / Dart · Zero-cost static engine · OpenAI · Claude · Gemini · CI · SARIF · JUnit · Web dashboard · MIT licensed

Installs via Composer with Laravel package auto-discovery. Only `tokenizer` is required; `pcntl` is optional for parallel analysis. Get started in one line: `php artisan codeguardian:analyze`.